SET 2

PART A — Short questions and answers

Q1. Describe a multivalued attribute using the phone numbers of a PATIENT as an example.

Answer: A multivalued attribute can have more than one value for one entity. A patient may have multiple phone numbers.

Q2. Explain the purpose of the NOT NULL constraint on Patient_Name.

Answer: NOT NULL requires a column to have a value; it prevents NULL entries.

Q3. Describe an update anomaly that arises when Doctor_Name is repeated in every appointment record.

Answer: An update anomaly occurs when repeated doctor details must be changed in many appointment rows. If one row is missed, inconsistent data remains.

Q4. Explain the purpose of the ON clause in a join between APPOINTMENT and DOCTOR.

Answer: The ON clause states the matching condition used to join rows from two tables.

Q5. Compare COUNT(*) and COUNT(column_name).

Answer: COUNT(*) counts rows. COUNT(column_name) counts only rows where that column is not NULL.

Q6. Describe the IN parameter mode of a PL/SQL procedure.

Answer: IN passes a value into a procedure. The procedure reads it but does not return a changed value through that parameter.

Q7. Explain why patient records with varying fields suit a document database.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Describe the BSON format used by MongoDB.

Answer: BSON is MongoDB's binary representation of JSON-like documents. It supports data types such as dates and numeric types.

Q9. Explain what the following MongoDB query returns: db.patients.find({Age: {$gt: 60}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain the role of a label on a node in Neo4j.

Answer: A label groups nodes by type, such as :Patient or :Doctor, making graph queries easier.

PART B — System solution

Scenario: A multi-speciality hospital maintains a Hospital Appointment Management System to store patient

1(a) ER design:

• **PATIENT**(PatientID, PatientName, Age, Gender, City); primary key: **PatientID**.

• **DOCTOR**(DoctorID, DoctorName, Specialization, ConsultationFee); primary key: **DoctorID**.

• **APPOINTMENT**(AppointmentID, PatientID, DoctorID, AppointmentDate, Diagnosis, FeePaid); primary key: **AppointmentID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE PATIENT (
  PatientID NUMBER PRIMARY KEY,
  PatientName VARCHAR2(100),
  Age NUMBER,
  Gender VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE DOCTOR (
  DoctorID NUMBER PRIMARY KEY,
  DoctorName VARCHAR2(100),
  Specialization VARCHAR2(100),
  ConsultationFee VARCHAR2(100)
);
```

```
CREATE TABLE APPOINTMENT (
  AppointmentID NUMBER PRIMARY KEY,
  PatientID NUMBER,
  DoctorID NUMBER,
  AppointmentDate DATE,
  Diagnosis VARCHAR2(100),
  FeePaid NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM APPOINTMENT x
JOIN PATIENT a ON x.PatientID = a.PatientID
JOIN DOCTOR b ON x.DoctorID = b.DoctorID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```